



Flowers Recognition

Using ResNet and CNN

Submitted By:

Sara Naif Aljohani - 445000162
Amal saeed Alzahrani - 445003599
Rola Bulayl Alsumlami-445007752

Data Modeling (DS3402T) Project
Dr. Tahani Alqurashi

Index

Abstract	3
Introduction	4
Data Exploration and Description	5
Methodology	7
• Classic CNN: From Scratch	
• ResNet: Fine Tuning	
Patterns Discovery and Results	11
Conclusion	13
References	15

Abstract

Write here,

Introduction

Image classification is one of the most important applications of data science and machine learning. It aims to enable intelligent systems to recognize image content and classify images into predefined categories with high accuracy. With the rapid development of deep learning techniques, it has become possible to handle image data that is highly variable and complex, which is often difficult for traditional machine learning methods to process effectively.

This project focuses on the classification of flower images using a dataset that contains a wide variety of images in terms of colors, shapes, lighting conditions, viewing angles, and backgrounds. Such high variability makes the classification task more challenging, as the models must learn to extract meaningful visual features and distinguish between visually similar classes.

To achieve the objectives of this project, both traditional machine learning and deep learning approaches were implemented and compared. A machine learning model was used as a baseline to evaluate performance, while two deep learning models were developed. The first deep learning model applied a fine-tuning (transfer learning) approach using a pre-trained model, whereas the second model was built from scratch using a convolutional neural network (CNN) architecture without relying on pre-trained weights.

The main goal of this comparison is to highlight the performance differences between traditional machine learning methods and deep learning techniques. In addition, the project examines the impact of using pre-trained models versus training models from scratch when dealing with high-variability image datasets. The results of this study help demonstrate the effectiveness of deep learning approaches in improving classification accuracy and addressing the challenges associated with complex visual data

Data Exploration and Description

The dataset utilized for this project consists of 4,242 digital images of flowers, categorized into five distinct classes: Chamomile, Dandelion, Rose, Sunflower, and Tulip.

The data was originally aggregated from diverse web sources (Flickr, Google Images, Yandex), introducing significant real-world variability in terms of lighting, angles, and backgrounds.

To assess the quality of the dataset before training, we conducted an Exploratory Data Analysis. As illustrated in Figure 1, the dataset is relatively well-balanced, with approximately 800 images per class. This balance is advantageous as it negates the need for synthetic oversampling techniques to prevent model bias toward a majority class.

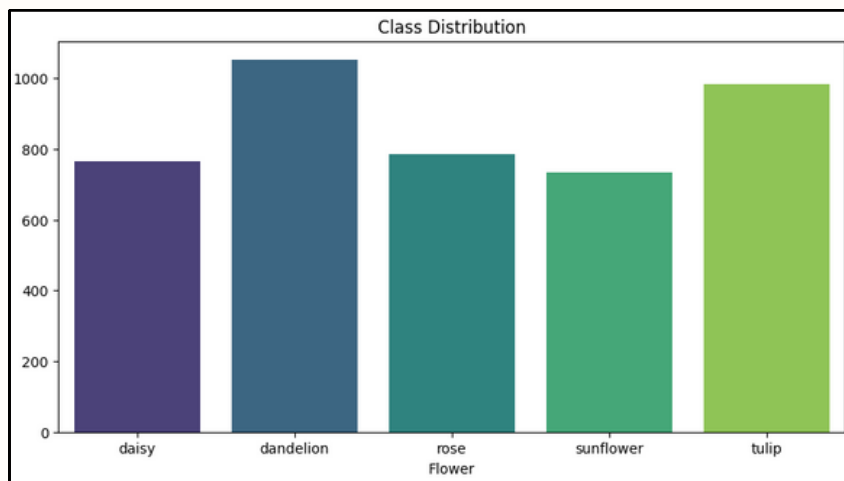


Figure 1: Class Distribution

However, the EDA revealed a significant preprocessing challenge regarding image resolution. As shown in the scatter plot analysis in Figure 2, the raw images exhibit high spatial heterogeneity, ranging from small thumbnails to high-resolution photographs. Since CNNs require fixed input dimensions, feeding this raw variable-sized data directly into the model is not feasible.

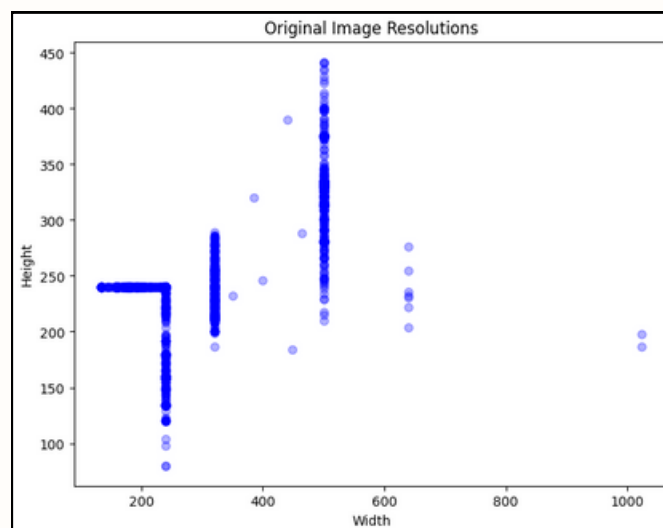


Figure 2: Images Resolutions

Furthermore, a qualitative inspection of random samples (Figure 3) confirmed variations in aspect ratios (landscape vs. portrait), .requiring a standardized cropping strategy (CenterCrop). This approach relies on the inherent composition of the dataset, where the subject—the flower—is typically positioned in the center of the frame.

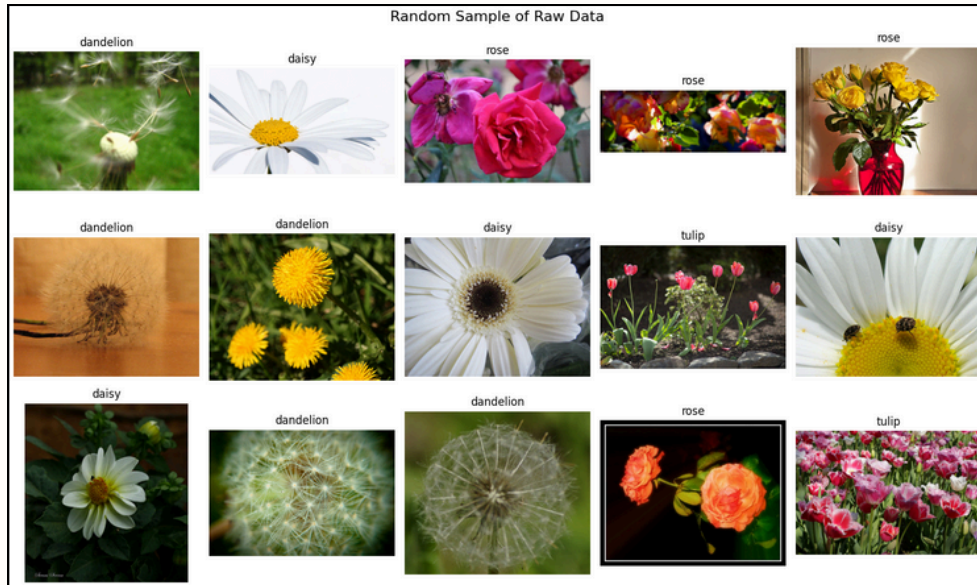


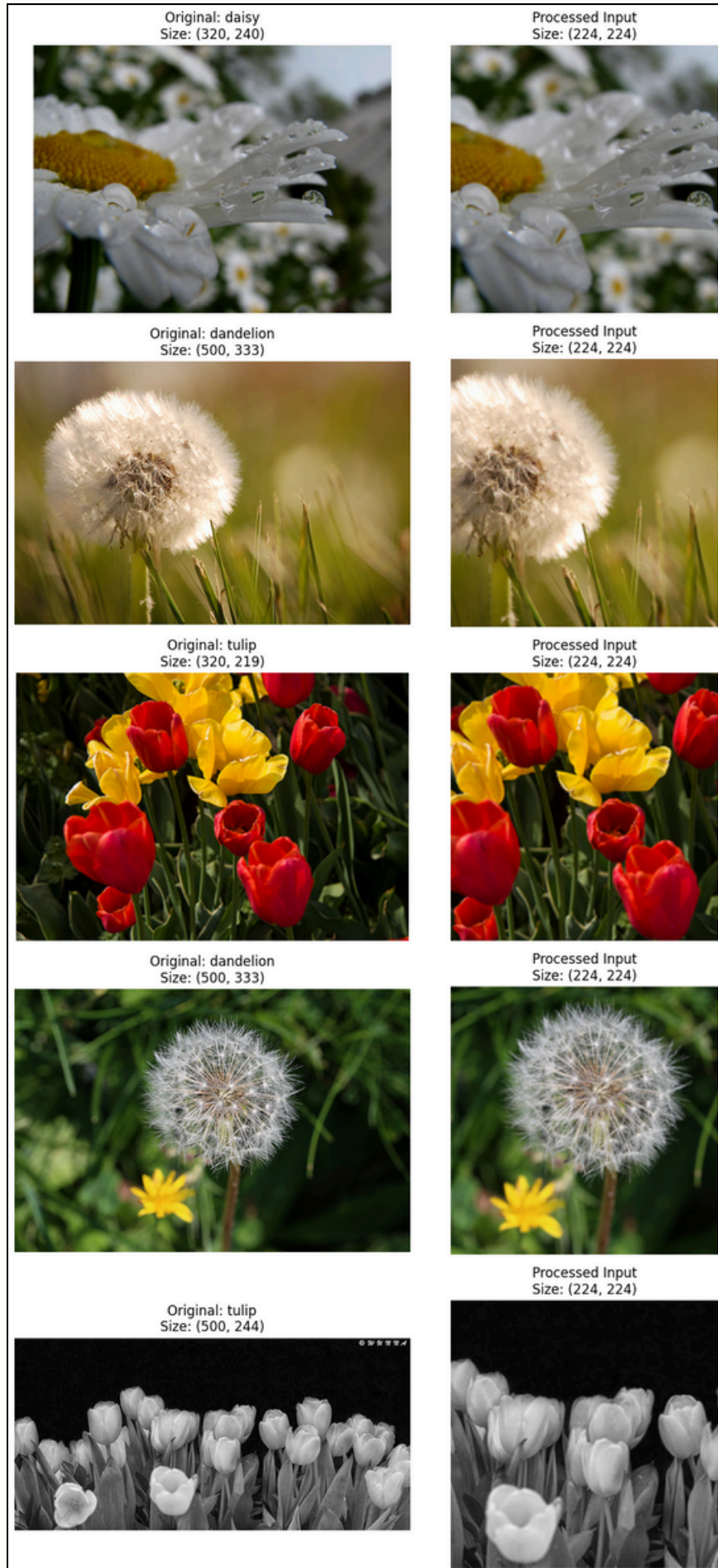
Figure 3: Random Sample of the Data

Data Preprocessing Strategy

To address the variability identified during exploration and to prepare the data for the **ResNet-18 architecture**, we implemented a robust preprocessing pipeline using torchvision.transforms. The following transformations were applied:

1. **Resizing and Cropping:** To handle the varying resolutions, all images were first resized to 255 pixels along the shorter edge. Subsequently, a Center Crop of 224×224 pixels was applied.
 - Justification: The decision to use 224×224 is derived from the architectural constraints of the pre-trained model. ResNet-18 weights (IMAGENET1K_V1) were trained on inputs of this specific dimensionality.⁽¹⁾ Deviating from this size would degrade the spatial feature extraction capabilities of the pre-trained weights.
2. **Normalization:** We applied channel-wise normalization using the mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225].
 - Justification: Neural networks converge faster and more stably when input data is centered and scaled. Furthermore, since we are utilizing Transfer Learning, it is statistically mandatory to normalize our new data to match the distribution of the original ImageNet dataset on which the model was pre-trained.
3. **Tensor Conversion:** Finally, the raw pixel data (Integers 0-255) was converted into PyTorch Tensors (Floating point 0.0-1.0), enabling efficient computation on the GPU.

By standardizing the input geometry and statistics, we ensured that the variability found during the EDA phase would not introduce noise into the training process.



*Figure 4: Random Sample of the Data
Before and After Resizing and Cropping.*

Methodology

write here

Methodology

Model Architecture Selection: ResNet-18

To address the challenge of high variability in the flower dataset, we employed a Convolutional Neural Network based on the ResNet-18 architecture. ResNet (Residual Network) was selected for its ability to train deep networks effectively using "skip connections," which mitigate the vanishing gradient problem.

We specifically chose the 18-layer variant (ResNet-18) rather than deeper models (e.g., ResNet-50 or ResNet-152) for the following reasons:

- **Prevention of Overfitting:** Our dataset contains 4,242 images. A massive model like ResNet-50 (25 million parameters) risks memorizing this relatively small dataset. ResNet-18 (~11 million parameters) offers sufficient complexity to learn features without excessive capacity for overfitting.
- **Efficiency:** ResNet-18 is computationally lightweight, allowing for faster training iterations on the available hardware (Google Colab T4 GPU).

Transfer Learning Strategy

Instead of training the network from scratch (random initialization), we utilized Transfer Learning. We loaded weights pre-trained on the ImageNet dataset (IMAGENET1K_V1), which contains 1.28 million images across 1,000 classes.

- **Feature Extraction:** The convolutional layers were frozen (`requires_grad=False`). This allows us to leverage the rich feature detectors (edges, textures, shapes) learned from ImageNet.
- **Fine-Tuning:** The final Fully Connected (FC) layer was replaced. The original 1,000-class output layer was discarded and substituted with a new linear layer mapping 512 input features to our 5 specific flower classes. Only this new layer was updated during training.

Methodology

Experimental Setup and Hyperparameters

The model was implemented using the PyTorch framework. Training was conducted on a Google Colab environment utilizing a Tesla T4 GPU for acceleration.

The training configuration was established as follows:

- **Loss Function:** CrossEntropyLoss was used, which is the standard objective function for multi-class classification problems.
- **Optimizer:** We employed the Adam optimizer with a learning rate of 0.001. Adam was chosen for its adaptive learning rate capabilities, which typically yield faster convergence than standard SGD.
- **Batch Size:** Set to 32 to balance memory usage and gradient stability.
- **Epochs:** The model was trained for 10 epochs, which was determined empirically to be sufficient for the validation accuracy to plateau without overfitting.
- **Data Split:** The dataset was partitioned into 80% Training, 10% Validation, and 10% Testing.

Evaluation Metrics

To objectively assess the performance of the model, we utilized the following metrics:

- **Accuracy:** The primary metric, calculated as the ratio of correctly predicted images to the total number of images in the validation/test sets.
- **Validation Loss:** Monitored during training to detect overfitting (i.e., if validation loss begins to increase while training loss decreases).
- **Confusion Matrix:** Used post-training to visualize misclassifications between specific pairs of flower classes (e.g., confusing a Rose with a Tulip).

Patterns Discovery and Results

Write here,

Conclusion

Write here,

References

[1] “resnet18 — Torchvision main documentation,” Pytorch.org, 2024.

<https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html>